CS5321 TEAM 35

Blocking Cross-Session Token Splicing in Token Exchange

Final Project Presentation

OAuth token exchange, delegation integrity, capability chains

> We study a delegation flaw in a vulnerable STS and show how a session-bound capability chain changes the execution result from forged acceptance to rejection.

- Threat: a stolen delegated token can be spliced with an attacker-controlled actor token.
- Observation: token validity alone is not enough; the delegation path must also be continuous.
- Contribution: enforce path integrity, session binding, signed hop chains, and replay-safe delegation.

AGENDA

1. Problem setting
2. Baseline flow
3. Vulnerable attack
4. Secure defense
5. Live demo
6. Results and cost

PROBLEM

Why Token Exchange Needs Path Integrity

Core invariant

`subject_token.aud == actor_token.act.sub`

SYSTEM MODEL

- User delegates through multiple agents using a shared STS.
- JWTs carry `sub`, `aud`, `act`, and `scope`.
- Each downstream exchange should preserve a continuous delegation path.

VULNERABILITY

- The vulnerable STS verifies tokens individually.
- It checks caller identity, but not whether the stolen token was issued to that caller.
- This enables cross-session token splicing.

SECURITY GOAL

- Reject any exchange that joins unrelated sessions.
- Prevent scope expansion and replay.
- Keep the mechanism lightweight enough for short delegation chains.

> The attack does not break cryptography. It exploits missing delegation-path validation inside the exchange service.

BASELINE

Normal Delegation Flow

- Alice first authenticates to Agent A.
- The STS issues an initial token for Agent A.
- Agent A delegates to Agent B with narrower scope.
- This is a valid exchange because the token presented to the STS was actually issued to the current actor.

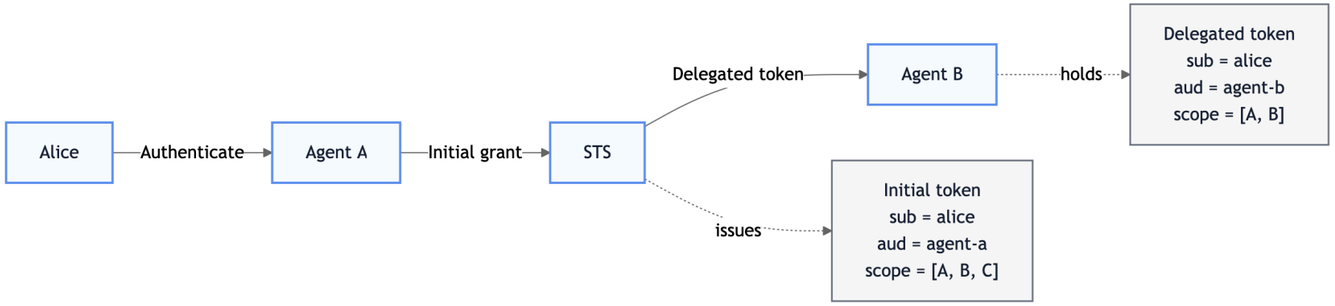

Figure from project report

ATTACK

Cross-Session Token Splicing

Same attack input
different system outcomes

- The attacker steals Agent B's delegated token.
- They combine it with their own actor token and submit both to the vulnerable STS.
- The caller check passes, but the path check is missing.
- The vulnerable STS mints a forged token and accepts a path that never existed.

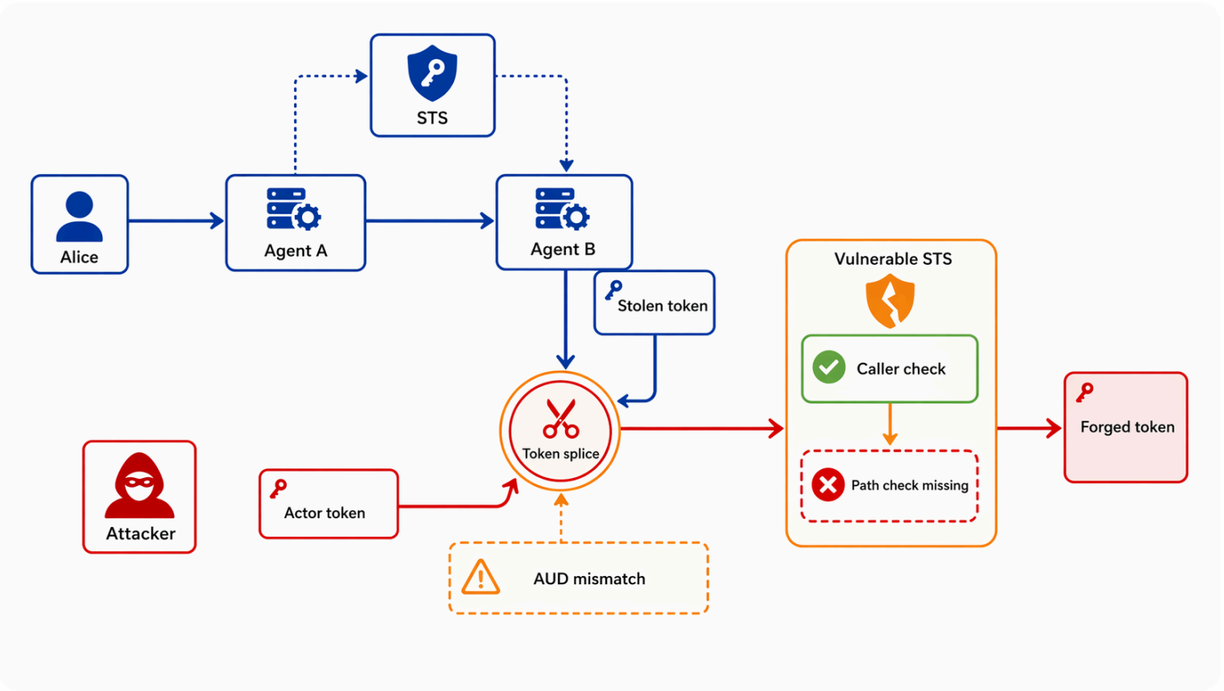

Figure: a stolen subject token and an attacker-controlled actor token are spliced into one forged exchange request.

DEFENSE

Session-Bound Capability Chain

Secure STS checks
caller, path, session, chain, scope, replay

- A direct path-integrity check rejects a caller that was never the intended recipient of the token.
- Each delegation hop is recorded as an STS-signed capability with a shared session identifier.
- The secure STS validates signatures, session continuity, hop order, path continuity, nonce freshness, and scope monotonicity.
- The same attack input now fails before a new token can be minted.

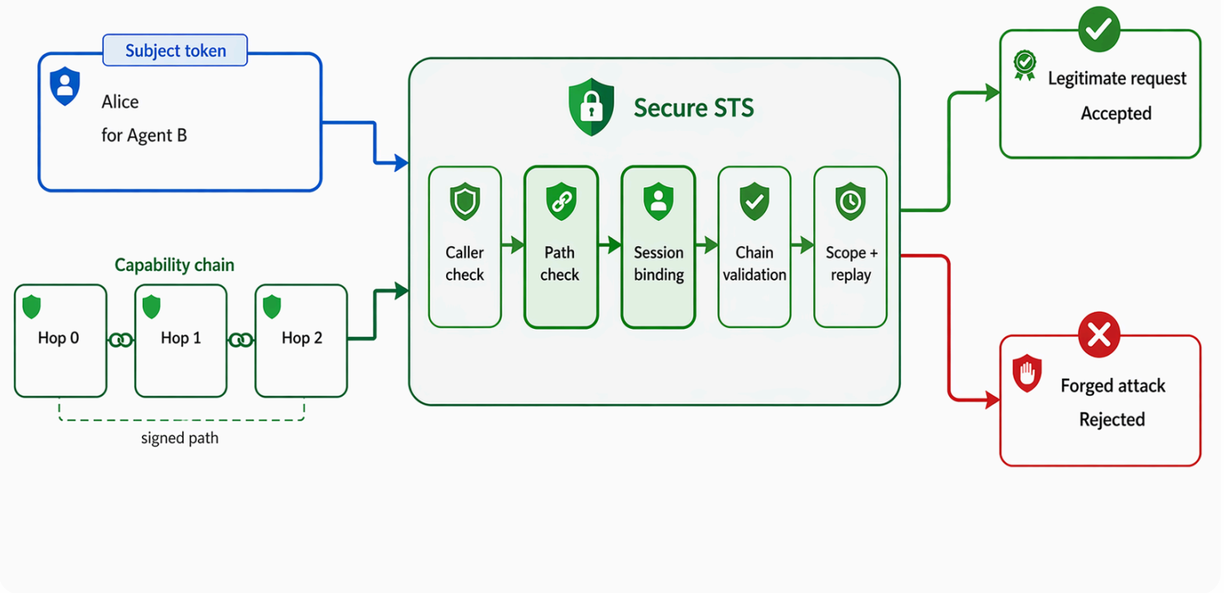

Figure: validation pipeline in the secure design. The attack is stopped when path integrity and session consistency are enforced.

LIVE DEMO

Run the Same Scenario in Code

Switch to terminal here

```
source .venv/bin/activate
python demo.py
```

DEMO FLOW

The script is enter-driven and shows the same attack input under two systems.

```
$ source .venv/bin/activate
$ python demo.py
Stage 1: Normal delegation flow
Stage 2: Same attack on Vulnerable STS
Stage 3: Same attack on Secure STS
Stage 4: Cost analysis
```

1. Show the legitimate prefix first so the audience sees the intended delegation path.
2. In the vulnerable system, the code really mints a forged token for the attacker.
3. In the secure system, the same kind of input is rejected by the validation pipeline.
4. Return to the slides for the result summary and measured overhead.

RESULT

Same Attack Input, Different Execution Result

Core comparison
vulnerable accepts, secure rejects

VULNERABLE STS

Forged token accepted

The caller check passes.

The path check is missing.

A new token is minted for the attacker, creating a forged path.

SECURE STS

Request rejected

The same kind of attack input reaches the secure exchange path.

Path integrity and session validation reject the request before minting.

The delegation chain is preserved as a security object, not just metadata.

REPLAY

Blocked by nonce freshness checks on the latest capability.

SCOPE ESCALATION

Blocked by requiring the requested scope to remain a subset of the previous hop.

TAMPERING

Blocked by HMAC-protected capability signatures and chain continuity checks.

Main empirical takeaway

EVALUATION

Cost Remains Small

Secure prototype
sub-millisecond across 1–5 hops

DEPTH	AVG LATENCY (MS)	CHAIN SIZE (CHARS)
1	0.04	580
2	0.09	872
3	0.13	1164
4	0.18	1456
5	0.23	1748

- Overhead grows roughly linearly with delegation depth because more chain state is validated.
- The measured exchange latency remains below one millisecond in this prototype.
- This makes the defense practical for short service-to-service delegation chains.
- The core benefit is stronger delegation integrity with modest runtime cost.

Measured on local prototype runs

CONCLUSION

What the Demo Establishes

Final takeaways
vulnerability, fix, cost

1. REAL VULNERABILITY

The vulnerable STS can be driven into minting a forged token using only valid, signed inputs.

2. REAL CHANGE IN CODE

Our secure design changes the execution result for the same attack pattern from acceptance to rejection.

3. MANAGEABLE COST

The defense adds measurable but still small overhead for short delegation chains.

> The project's central claim is not just that the attack can be explained, but that delegation integrity can be enforced in code with a clear and practical validation path.